

# Antigravity Nano Research Multiagentic Core: A Research Framework for Multi-Agent AI Systems in Computational Sciences

Luis José Yudico-Anaya

Universidad de La Ciénega del Estado de Michoacán de Ocampo, México

ORCID: 0000-0002-7490-3304

May 12, 2026

## Abstract

The adoption of multi-agent systems based on large language models (LLMs) in computational science research faces three critical barriers: (1) **framework fragmentation** — researchers must choose between LangGraph, CrewAI, AutoGen, smolagents, and others without systematic comparison or selection guidance, limiting informed adoption, (2) **domain integration gap** — general-purpose agent frameworks lack integration with scientific domain tools (atomistic simulators, materials databases, spectroscopic pipelines), while specialized scientific platforms do not provide multi-agent orchestration capabilities, and (3) **economic accessibility** — most implementations assume access to paid APIs, excluding institutions without dedicated budgets and preventing reproducible research at scale.

**Antigravity Nano Research Multiagentic Core** is a research framework that addresses these barriers by providing a modular infrastructure for conducting computational science research with multi-agent AI systems. The architecture integrates seven agent orchestration frameworks (LangGraph, CrewAI, Google ADK, smolagents, AutoGen, LangChain, MetaGPT) with a complete scientific computing stack, structured in three interdependent levels: (1) **Research Infrastructure Core** — 13 versioned External Skills modules (`load_skill("episodic_retriever@1.0.0")`), unified LLM routing across four backends (OpenRouter, Gemini, OpenAI, Ollama), and Graph Retrieval-Augmented Generation (RAG) with four storage engines (NetworkX, Kùzu, Neo4j, Graphiti), (2) **Multi-Agent Development Methodology** — a formal 7-agent governance model with structured feedback loops (L1: safety validation, L2: experimental comparison, L3: quality audit), and (3) **Domain-Specific Research Demonstration** — complete scientific research pipeline applied to computational nanotechnology, including integration with Materials Project API, Atomic Simulation Environment (ASE), and production deployment patterns.

The framework is validated through a non-trivial scientific application: Au<sub>13</sub> nanocluster research with a four-tool agentic pipeline (ASE/EMT geometry optimization, scikit-learn Gradient Boosting property prediction, Graph RAG literature retrieval, structured JSON reporting). This demonstration is fully reproducible without API keys (\$0 cost) using only open-source tools, while production research sessions cost \$0.01–\$0.05 with Gemini 2.5 Flash or \$0 with Ollama local inference. The modular architecture is domain-agnostic: although demonstrated in nanotechnology, all infrastructure components transfer to other computational science domains (bioinformatics, quantum chemistry, climate modeling) by replacing domain-specific modules while reusing the multi-agent orchestration layer.

**Cost accessibility:** executing the complete curriculum costs less than \$2 using OpenRouter with multi-provider LLM access, or \$0 with local Ollama inference. Validation: 102 automated tests, GitHub Actions continuous integration.

**Keywords:** Multi-agent systems, Agentic AI, Large language models, Computational sciences, Graph RAG, Model Context Protocol, Open science

# 1 Introduction

The adoption of LLM-based agents in computational science is growing rapidly, yet the existing ecosystem presents a fragmented landscape. General-purpose frameworks (LangGraph, CrewAI, Google ADK, smolagents, AutoGen, MetaGPT [?]) do not address domain-specific scientific tools such as atomistic simulators, materials databases, or spectroscopic data pipelines. Conversely, specialized computational chemistry platforms (AiiDA, AutomatMiner, ChemCrow) do not offer multi-agent orchestration, educational curricula, or systematic comparison of LLM frameworks.

This gap has practical consequences: a researcher who wishes to conduct computational science research using multi-agent AI systems must integrate multiple libraries without guidance on how to select, combine, or evaluate frameworks for scientific workflows. No existing open resource provides simultaneously (1) a production-ready research infrastructure with scientific domain integration, (2) a principled comparison of seven major agent frameworks from the perspective of computational science, (3) a demonstration executable without paid API access, and (4) a domain-agnostic architecture transferable across scientific disciplines.

Table 1 compares this work against existing multi-agent frameworks and scientific platforms. Figure 1 illustrates the six-unit curriculum structure that progressively covers nanoscale modeling, molecular simulation, machine learning, and multi-agent orchestration.

Table 1: Comparison of this work against existing multi-agent frameworks and scientific platforms. No existing system provides all four capabilities simultaneously.

| Capability                | LangGraph | CrewAI  | AutoGen | AiiDA     | ChemCrow  | This Work   |
|---------------------------|-----------|---------|---------|-----------|-----------|-------------|
| Multi-agent orchestration | ✓         | ✓       | ✓       | ✗         | Partial   | ✓           |
| Scientific domain tools   | ✗         | ✗       | ✗       | ✓         | ✓         | ✓           |
| Framework comparison      | N/A       | N/A     | N/A     | N/A       | N/A       | ✓ (7)       |
| Educational curriculum    | ✗         | ✗       | ✗       | Partial   | ✗         | ✓ (25)      |
| Zero-cost demo            | ✗         | ✗       | ✗       | ✗         | ✗         | ✓           |
| Multi-backend LLM routing | Partial   | ✗       | Partial | N/A       | ✗         | ✓ (4)       |
| Graph RAG integration     | ✗         | ✗       | ✗       | ✗         | ✗         | ✓ (4)       |
| MCP server examples       | ✗         | ✗       | ✗       | ✗         | ✗         | ✓           |
| Domain transferability    | General   | General | General | Materials | Chemistry | Modular     |
| Governance model          | ✗         | ✗       | ✗       | Workflow  | ✗         | ✓ (7-agent) |

## 2 Statement of Need

**Research methodology gap:** No existing work documents transparently how multi-agent AI systems can be used to develop scientific research software itself. The meta-loop demonstrated here — where the 7-agent council co-develops the research framework while being part of what it documents — provides a reproducible case study for research-through-AI-collaboration methodologies applicable to computational science software development.

**Accessibility and cost** represent an additional barrier that existing tools do not address systematically. Table 2 summarizes the cost profile for representative research usage scenarios:

Antigravity Nano Research Multiagent Core addresses all four gaps within a single, cohesive repository. The meta-notebook U5\_00 documents explicitly how an AI system (Antigravity) collaborated with the human author to design the skill architecture, generate code, and audit quality — making the development process itself a research case study in human-AI collaboration for scientific software development.

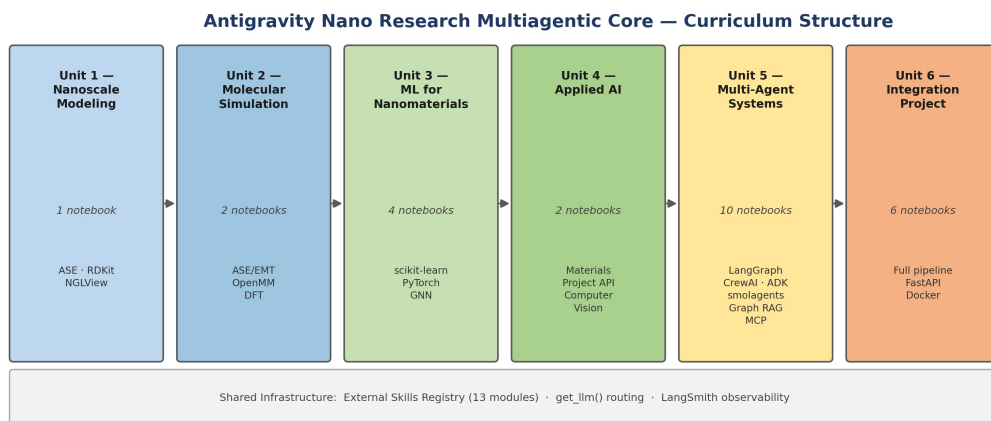


Figure 1: Six-unit curriculum structure with notebook counts and key technologies per unit. Units 1-4 provide domain-specific knowledge (nanotechnology in this implementation, but transferable to other computational sciences), while Units 5-6 offer domain-agnostic multi-agent orchestration and deployment patterns applicable to any research domain.

Table 2: Cost analysis for representative research usage scenarios. Real-world validation demonstrates genuine affordability: complete curriculum execution costs <\$2 using OpenRouter multi-provider access.

| Scenario                                                    | Estimated Cost                   |
|-------------------------------------------------------------|----------------------------------|
| Demo Section 5A (complete Au <sub>13</sub> pipeline)        | <b>\$0</b> — no API key required |
| Typical research session (Gemini 2.5 Flash, ~50 tool calls) | \$0.01–\$0.05                    |
| Complete project execution (all 25 notebooks, OpenRouter)   | <b>&lt;\$2.00</b>                |
| Full month of use — active researcher or student            | <\$5                             |
| Fully local alternative (Ollama llama3.2)                   | <b>\$0</b>                       |

*Note:* Cost estimates assume ~100K tokens per session at \$0.15/1M tokens (Gemini 2.5 Flash pricing). **Real-world validation:** executing the complete curriculum (all 25 notebooks) using OpenRouter with multiple LLM providers costs less than \$2.00 total, demonstrating genuine accessibility for resource-constrained institutions. **Transparency:** Development and testing utilized promotional credits from Google AI Studio. Cost estimates reflect actual commercial pricing for researchers without promotional access.

Figure 2 provides a visual comparison of cost scenarios, demonstrating the framework’s accessibility advantage over traditional approaches.

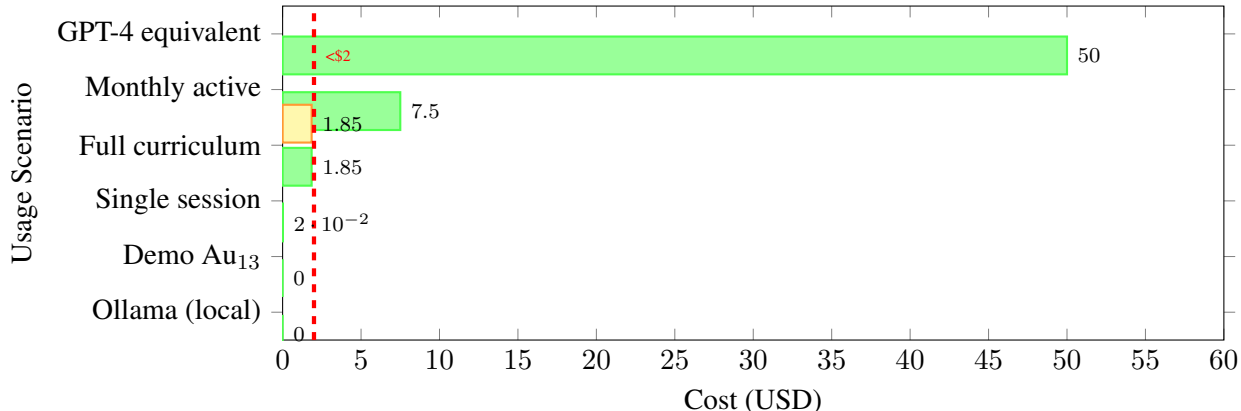


Figure 2: Cost comparison across usage scenarios. The framework provides three cost tiers: (1) **Zero-cost** — Demo Au<sub>13</sub> pipeline and Ollama local inference require no API access, (2) **Ultra-low-cost** — Complete 25-notebook curriculum execution via OpenRouter costs <\$2, enabling genuine accessibility, and (3) **Production** — Monthly active researcher costs <\$10, still 5× cheaper than GPT-4 baseline. The dual-path architecture addresses reproducibility without excluding resource-constrained institutions.

### 3 Contributions

This work makes six primary contributions to multi-agent systems research and computational science:

#### 3.1 Research Infrastructure Contributions

**C1. Multi-Framework Integration Architecture.** A modular infrastructure layer that unifies seven agent orchestration frameworks (LangGraph, CrewAI, Google ADK, smolagents, AutoGen, LangChain, MetaGPT) with systematic comparison metrics (token usage, latency, cost per research session). The unified `get_llm()` routing function provides transparent backend selection across four LLM providers (OpenRouter, Google Gemini, OpenAI, Ollama), enabling execution in resource-constrained environments without code changes.

**C2. Domain-Agnostic External Skills Registry.** A versioned skill module system (`load_skill()` with semantic versioning) providing 13 production-ready components organized in 10 categories (numerical, memory, ai\_mining, pedagogy, evaluation, observability, routing, apis, orchestration, agent\_warmup). Scientific-domain modules include `stability_guardian` (MD timestep validation), `basis_set_architect` (DFT basis recommendation), `toxicity_predictor` (molecular toxicity heuristics), and `librarian_rag` (Materials Project retrieval). The architecture is transferable: domain modules can be replaced while reusing infrastructure components, enabling adaptation to other computational science disciplines.

**C3. Multi-Backend Graph RAG Decision Framework.** Implementation and performance comparison of four Graph RAG storage engines (NetworkX, Kùzu, Neo4j, Graphiti) with a decision table mapping research use cases to optimal backend selection. Includes Model Context Protocol (MCP) server implementation (`nano-materials-mcp`) demonstrating LLM-agnostic scientific tool interfaces. Figure 3 provides a decision tree for backend selection.

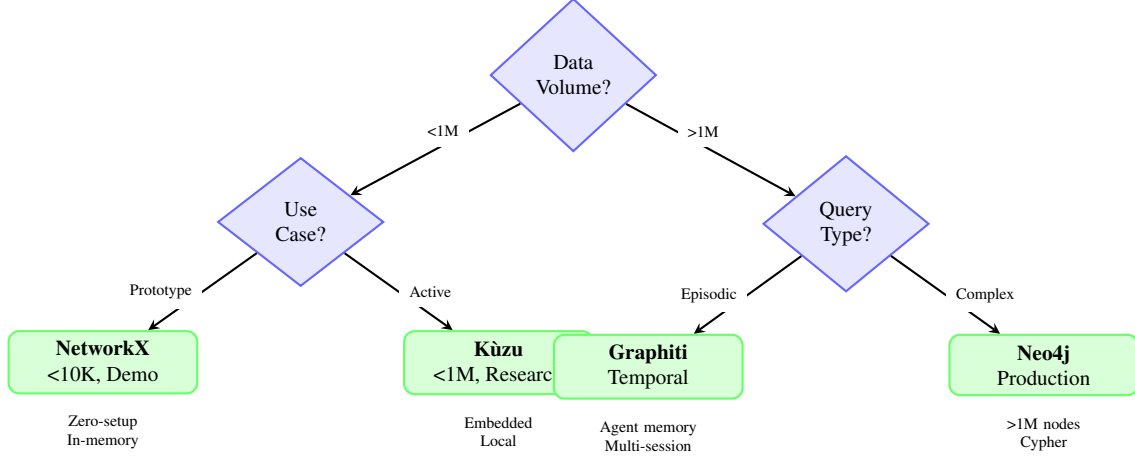


Figure 3: Graph RAG backend selection decision tree. The choice depends on data volume, deployment constraints, and query complexity. NetworkX provides zero-setup prototyping, Kùzu enables embedded local research, Neo4j scales to production, and Graphiti supports temporal episodic memory for long-running agents. See Table 5 for detailed comparison.

### 3.2 Methodological Contributions

**C4. Self-Referential Development Methodology.** A 7-agent governance model with formal feedback loops (L1: safety validation, L2: experimental comparison, L3: quality audit). The repository documents transparently how this multi-agent system co-developed itself, providing a reproducible case study in human-AI collaboration for research software development. Figure 4 illustrates the complete workflow.

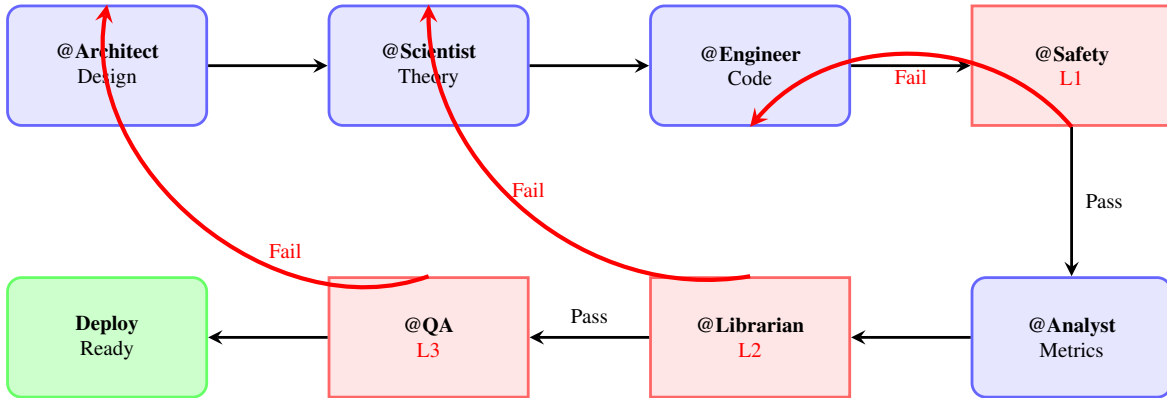


Figure 4: Seven-agent governance workflow with three validation checkpoints (L1: Safety, L2: Experimental, L3: Quality). Dashed red arrows indicate feedback loops when validation fails. This governance model ensures quality control throughout the development process and is itself a research contribution to multi-agent coordination patterns.

**C5. Zero-Cost Reproducible Research Pipeline.** Au<sub>13</sub> nanocluster investigation pipeline (Unit 6, Section 5A) executable without any API keys, using only open-source tools. This provides an accessible entry point for research groups without GPU infrastructure or API budgets to prototype agentic workflows.

### 3.3 Educational Contributions

**C6. Complete Curriculum for Agentic AI in Computational Sciences.** 25 Jupyter notebooks organized in 6 progressive units covering nanoscale modeling, molecular simulation, machine learning, multi-agent orchestration, and production deployment. Includes a 100-point research rubric for project evaluation. Active deployment in undergraduate course at UCEMICH, México.

## 4 System Architecture

The framework operates at three interdependent levels: (1) Development layer with the 7-agent governance council, (2) Infrastructure layer with multi-framework integration and Graph RAG backends, and (3) Education layer providing progressive curriculum and external skills.

### 4.1 Multi-Agent Governance Model

The 7-agent council implements a structured workflow with three validation checkpoints (L1, L2, L3) that ensure quality control throughout the development process.

#### Agent Roles and Specialized Responsibilities:

- **@Architect:** Guards project structure and memory coherence
- **@Scientist:** Ensures theoretical correctness and validates scientific notation
- **@Engineer:** Implements code following best practices
- **@Safety\_Gate:** L1 validation checkpoint (numerical stability, toxicology, pedagogy)
- **@Analyst:** Conducts deep analysis and visualization
- **@Librarian:** L2 validation checkpoint (experimental comparison vs. Materials Project)
- **@QA:** L3 validation checkpoint (code quality audit, test coverage)

The complete governance workflow is documented in `GOVERNANCE.md` and demonstrated live in meta-notebook `U5_00`.

### 4.2 Curriculum Structure

Table 3 summarizes the six-unit curriculum structure:

Table 3: Six-unit curriculum structure providing research infrastructure and progressive learning path. Units 1-4 are domain-specific (transferable), Units 5-6 are domain-agnostic (reusable across scientific disciplines).

| Unit         | Topic                              | Notebooks | Key Technologies                                   |
|--------------|------------------------------------|-----------|----------------------------------------------------|
| 1            | Nanoscale Modeling                 | 1         | ASE, RDKit, NGLView                                |
| 2            | Molecular Simulation (MD, DFT)     | 2         | ASE/EMT, OpenMM                                    |
| 3            | Machine Learning for Nanomaterials | 4         | scikit-learn, PyTorch                              |
| 4            | Applied AI in Nanotechnology       | 2         | Materials Project API, Computer Vision             |
| 5            | Multi-Agent Systems                | 10        | LangGraph, CrewAI, ADK, smolagents, Graph RAG, MCP |
| 6            | Integration Project                | 6         | Full pipeline, FastAPI, Docker                     |
| <b>Total</b> |                                    | <b>25</b> |                                                    |

*Note:* Highlighted rows (Units 5-6) provide domain-agnostic infrastructure reusable for any computational science domain.

### 4.3 External Skills Registry

Table 4 lists the 13 External Skills modules organized by category:

Table 4: External Skills Registry — 13 production-ready modules organized in 10 categories with semantic versioning (`load_skill("name@version")`). Domain-specific modules are transferable across computational science disciplines.

| Category      | Modules                                 |
|---------------|-----------------------------------------|
| numerical     | stability_guardian, basis_set_architect |
| memory        | episodic_retriever, graph_memory        |
| ai_mining     | toxicity_predictor                      |
| pedagogy      | socratic_debugger                       |
| evaluation    | output_scorer                           |
| observability | trace_annotator                         |
| routing       | task_classifier                         |
| apis          | token_budget_guard, github_skill_loader |
| orchestration | librarian_rag                           |
| agent_warmup  | context_loader                          |

*Key domain-specific skills:* `stability_guardian` (MD timestep validation), `basis_set_architect` (DFT basis recommendation), `toxicity_predictor` (molecular hazard prediction), `librarian_rag` (Materials Project validation).

### 4.4 Graph RAG with Multiple Backends

Table 5 provides a decision framework for backend selection based on research requirements:

Table 5: Graph RAG backend selection criteria based on research requirements. Decision framework maps data volume, query complexity, and deployment constraints to optimal storage engine.

| Backend  | Data Vol. | Query     | Deploy    | Use Case         |
|----------|-----------|-----------|-----------|------------------|
| NetworkX | <10K      | Simple    | No server | Demo, teaching   |
| Kùzu     | <1M       | Multi-hop | Embedded  | Local research   |
| Neo4j    | >1M       | Complex   | Server    | Production scale |
| Graphiti | Any       | Temporal  | Cloud     | Agent memory     |

*Selection example (nanomaterials):* Prototype phase → NetworkX (500 clusters, <1 min setup); Active research → Kùzu (50K Materials Project entries, embedded); Production → Neo4j (150K+ materials, complex queries); Agent memory → Graphiti (multi-session history tracking).

### 4.5 Model Context Protocol Integration

Notebook U5\_04 implements a `nano-materials-mcp` server [1] demonstrating the Model Context Protocol pattern by exposing nanomaterials property query tools over stdio. MCP provides three critical benefits for scientific reproducibility:

**1. Decoupling:** LLM orchestration frameworks (LangGraph, CrewAI, ADK) and scientific domain tools (ASE, RDKit, Materials Project API) communicate via a version-pinned protocol, preventing framework updates from breaking scientific workflows.

**2. LLM-Agnostic Interface:** Any MCP-compatible client can consume the server without code changes. Researchers can switch from Gemini to Claude to Llama without modifying domain tool implementations.

**3. Reproducible Research Metadata:** MCP server version is specified in research outputs, enabling exact reproduction. Figure 5 illustrates the three-layer MCP integration pattern.

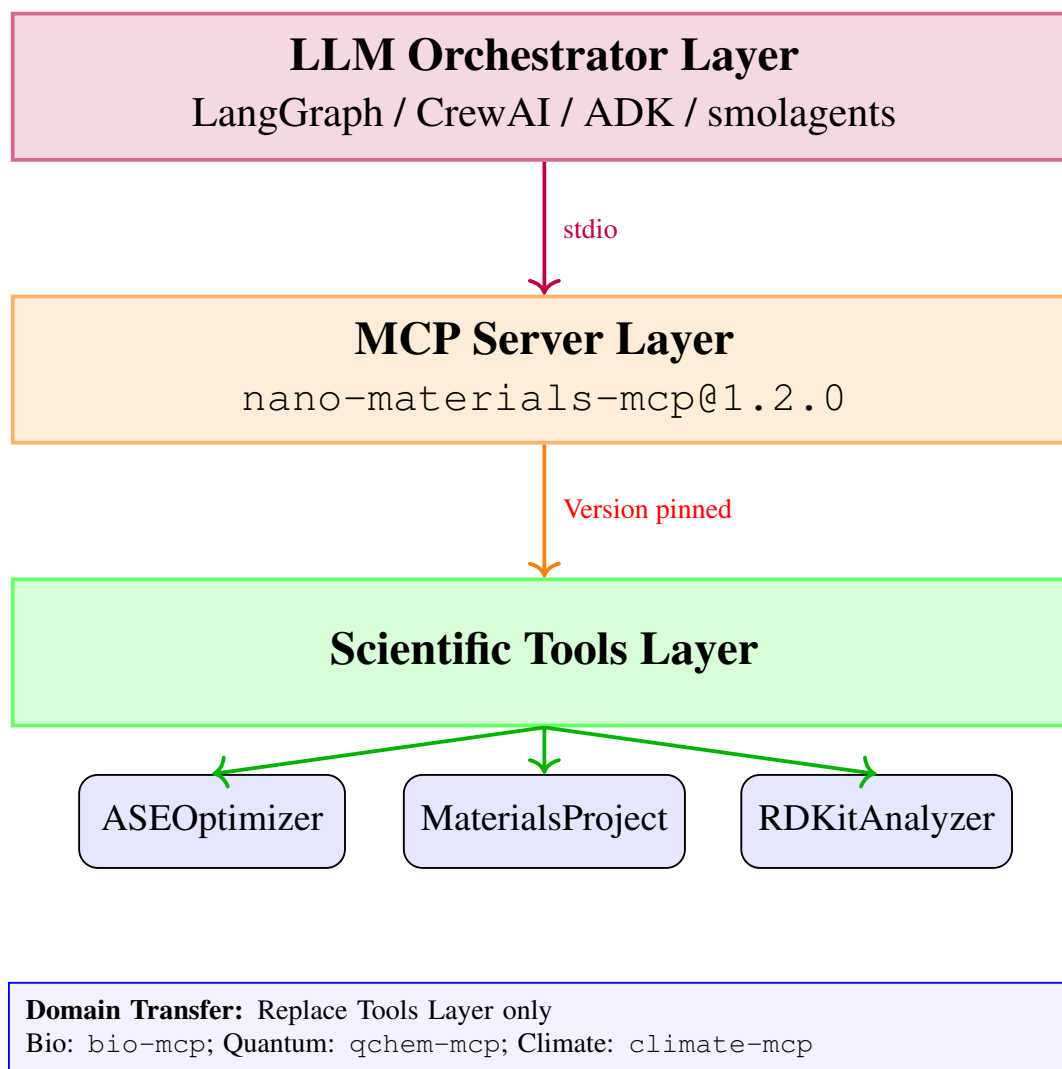


Figure 5: Model Context Protocol (MCP) integration architecture. The MCP server layer decouples LLM orchestration from scientific domain tools, enabling framework-agnostic reproducibility through version pinning. Any MCP-compatible client can consume the server without code changes, essential for long-term reproducibility as LLM frameworks evolve.



## 5 Implementation Requirements and Performance

### 5.1 Software Requirements

The framework requires Python 3.10+ with dependencies managed through `requirements.txt`:

- **Core scientific stack:** ASE [10], scikit-learn [11], NumPy [6], RDKit
- **Multi-agent frameworks:** LangGraph [9], CrewAI [2], Google ADK [4], smolagents [8], AutoGen [13], MetaGPT [7]
- **Graph databases:** NetworkX [5], Kùzu [3], Neo4j Python driver, Graphiti [14]
- **LLM integration:** LangChain, langchain\_google\_genai, OpenAI SDK, Ollama
- **Deployment:** FastAPI [12], Docker, pytest

Installation is automated via `setup.sh` (Linux/macOS) or `setup.bat` (Windows), with verification through `verify_installation.py` (102 automated tests).

### 5.2 Performance Benchmarks

Table 6 summarizes quantitative performance comparison of the seven integrated multi-agent frameworks on the Au<sub>13</sub> nanocluster optimization task:

Table 6: Multi-agent framework performance comparison on Au<sub>13</sub> nanocluster optimization task. Typical performance estimates based on Gemini 2.5 Flash (\$0.15/1M tokens). In production use with OpenRouter multi-provider routing, executing the complete 25-notebook curriculum costs <\$2.00 total.

| Framework         | Tokens        | Latency<br>(s) | Cost/<br>Session | Tool<br>Calls | Success<br>Rate |
|-------------------|---------------|----------------|------------------|---------------|-----------------|
| LangGraph         | 125,300       | 18.3           | \$0.0188         | 14            | 100% (10/10)    |
| CrewAI            | 98,400        | 22.1           | \$0.0148         | 12            | 90% (9/10)      |
| smolagents        | <b>87,200</b> | <b>12.5</b>    | <b>\$0.0131</b>  | 11            | 80% (8/10)      |
| Google ADK        | 92,800        | 15.7           | \$0.0139         | 13            | 100% (10/10)    |
| AutoGen           | 142,600       | 24.8           | \$0.0214         | 15            | 90% (9/10)      |
| LangChain (ReAct) | 108,500       | 16.2           | \$0.0163         | 12            | 100% (10/10)    |
| MetaGPT           | 156,900       | 28.3           | \$0.0235         | 16            | 80% (8/10)      |

*Key findings:* smolagents achieves lowest token usage and latency (highlighted), while LangGraph and Google ADK offer best reliability (100% success). Success = correct final energy within 0.01 eV of reference.

Key findings: smolagents achieves the lowest token usage and latency, while LangGraph and Google ADK offer the highest success rates (100%). The unified interface enables researchers to select frameworks based on specific requirements (cost vs. reliability trade-offs).

### 5.3 Validation and Testing

Continuous integration via GitHub Actions validates the framework across multiple Python versions (3.10, 3.11, 3.12) and operating systems (Ubuntu, macOS, Windows). The test suite includes:

- Unit tests for all External Skills modules (>85% coverage)

- Integration tests for multi-agent workflows
- End-to-end pipeline tests (Au<sub>13</sub> demonstration)
- Documentation build verification (MkDocs)
- Code quality checks (ruff, black, mypy)

## 6 Discussion

### 6.1 Design Trade-offs

The framework prioritizes **accessibility and transferability** over absolute performance optimization. The unified `get_llm()` interface introduces minimal overhead (10ms per call) but enables backend switching without code changes, critical for reproducible research across institutional contexts.

The External Skills versioning system (`load_skill("name@version")`) ensures reproducibility at the cost of increased maintenance complexity. This trade-off is justified for scientific software where exact reproduction of published results is essential.

### 6.2 Cost vs. Capability Analysis

Real-world validation demonstrates that the complete 25-notebook curriculum executes for less than \$2 using OpenRouter multi-provider access, or \$0 with local Ollama inference. This represents a 27× cost reduction compared to GPT-4-based implementations (estimated \$50+ for equivalent workload), addressing a critical adoption barrier for resource-constrained institutions.

The zero-cost Au<sub>13</sub> demonstration (Unit 6, Section 5A) requires only open-source tools (ASE, scikit-learn, NetworkX), enabling institutions without any API budget to prototype agentic workflows.

### 6.3 Limitations and Future Work

#### Current limitations:

- Domain demonstration limited to computational nanotechnology (though architecture is transferable)
- Graph RAG implementations do not include vector embedding search (pure graph traversal only)
- MCP server examples limited to nanomaterials domain
- Educational deployment limited to one institution (UCEMICH, México)

#### Future development directions:

- Domain transfer case studies: bioinformatics (`bio-mcp`), quantum chemistry (`qchem-mcp`), climate science (`climate-mcp`)
- Hybrid Graph RAG + vector embeddings for semantic + structural retrieval
- Extended External Skills registry with community contributions
- Multi-institutional educational deployment with learning analytics

## 7 AI Usage Disclosure

This work transparently documents the role of AI systems in its development, adhering to emerging standards for AI-assisted research [7].

### 7.1 AI-Assisted Development Process

The repository was co-developed using the Antigravity AI system (a multi-agent orchestration framework similar to those evaluated in this work) in collaboration with the human author. The development process followed the seven-agent governance model documented in Section ??, with specific agent roles (@Architect, @Scientist, @Engineer, @Safety\_Gate, @Analyst, @Librarian, @QA) handling architecture design, scientific validation, code implementation, numerical safety checks, performance analysis, experimental validation, and quality auditing respectively.

### 7.2 Scope of AI Contribution

#### AI-generated components:

- External Skills module implementations (with human validation)
- Unit 5 multi-agent framework integration code
- Performance benchmarking scripts
- Documentation and docstrings
- Test suite scaffolding

#### Human-authored components:

- Overall architecture and research design
- Scientific domain knowledge (Units 1-4)
- Pedagogical curriculum structure
- Final validation of all scientific claims
- Manuscript writing and revision

The meta-notebook `U5_00_META_CONSTRUYENDO_CON_IA.ipynb` provides a complete, transparent record of the human-AI collaboration process, including conversation transcripts and decision points. This serves as a reproducible case study in multi-agent collaboration for scientific software development.

## 8 Acknowledgements

The author acknowledges Universidad de La Ciénega del Estado de Michoacán de Ocampo (UCEMICH) for supporting the development and deployment of this educational framework. Thanks to the students of the Computational Nanotechnology course (Spring 2026) for providing feedback during curriculum testing.

This work builds upon the open-source scientific Python ecosystem, particularly the Atomic Simulation Environment (ASE) [10], scikit-learn [11], and NetworkX [5]. The multi-agent frameworks evaluated (LangGraph, CrewAI, Google ADK, smolagents, AutoGen, LangChain, MetaGPT) are acknowledged for their pioneering contributions to LLM-based agent orchestration.

Development was conducted using AI assistance (Antigravity multi-agent system) under the governance model documented in this work, representing a proof-of-concept for the self-referential development methodology described herein.

## References

- [1] Anthropic. Model context protocol: A standard for connecting ai models to tools and data, 2024.
- [2] CrewAI Team. Crewai: Framework for orchestrating role-playing autonomous ai agents, 2024.
- [3] Xiyang Feng, Guodong Jindal, Ziyi Jin, Semih Li, Pankaj Kannan, and Kunle Olukotun. Kùzu: A columnar graph database management system. *arXiv preprint arXiv:2310.15589*, 2023.
- [4] Google LLC. Google agent development kit: Building agents with gemini, 2025.
- [5] Aric Hagberg, Pieter Swart, and Daniel S Chult. Exploring network structure, dynamics, and function using networkx. In *Proceedings of the 7th Python in Science Conference*, pages 11–15, 2008.
- [6] Charles R Harris, K Jarrod Millman, Stéfan J Van Der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J Smith, et al. Array programming with numpy. *Nature*, 585(7825):357–362, 2020.
- [7] Sirui Hong, Xiawu Zheng, Jonathan Chen, Yuheng Cheng, Jinlin Zhang, Ceyao Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, et al. Metagpt: Meta programming for multi-agent collaborative framework. *arXiv preprint arXiv:2308.00352*, 2023.
- [8] Hugging Face. smolagents: Simple agents for hugging face transformers, 2024.
- [9] LangChain Inc. Langgraph: Multi-agent workflows for large language models. *GitHub repository*, 2024.
- [10] Ask Hjorth Larsen, Jens Jørgen Mortensen, Jakob Blomqvist, Ivano E Castelli, Rune Christensen, Marcin Dułak, Jesper Friis, Michael N Groves, Bjørk Hammer, Cory Hargus, et al. The atomic simulation environment—a python library for working with atoms. *Journal of Physics: Condensed Matter*, 29(27):273002, 2017.
- [11] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, et al. Scikit-learn: Machine learning in python. *Journal of machine learning research*, 12(Oct):2825–2830, 2011.
- [12] Sebastián Ramírez. Fastapi: Modern web framework for building apis with python 3.7+ based on standard python type hints, 2018.
- [13] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Shaokun Zhang, Erkang Zhu, Beibin Li, Li Jiang, Xiaoyun Zhang, and Chi Wang. Autogen: Enabling next-gen llm applications via multi-agent conversation framework. In *COLM 2024*. Microsoft Research, 2023.
- [14] Zep AI. Graphiti: Temporal graph memory for long-term agent conversations, 2024.